

Islamic University of Technology

Course Code : EEE 4308
Course Name : Digital Electronics Lab
Project Name : SAP-1 Architecture-based 8-bit Computer Design and Implementation.
Phase No. : C

Team : Byte_Masters

Name	Student ID
Irfan Hossain Rafi	230021331
Rafin Islam	230021323
Sanajid Arham Bakar	230021311
Raad al Hasan	230021322
Nazifa Noushin	230021117
Samiha Alam Purna	230021127

Supervisor:
Mr. Md. Rahib-Bin-Hossain

Junior Lecturer, Electrical & Electronic Engineering Department

Islamic University of Technology

Phase C of the **SAP-1 Architecture-based 8-bit Computer Design and Implementation** project focuses on constructing the central processing and control mechanisms of the computer. While previous phases may have established the structural foundation, Phase C integrates the brain.

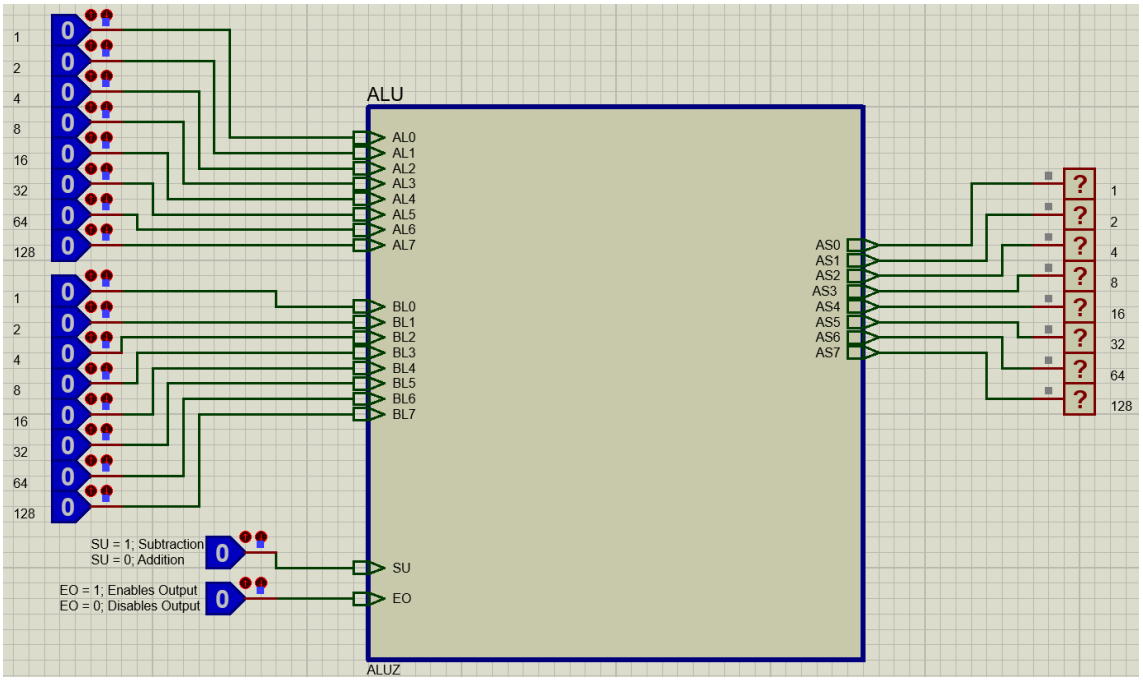
This phase details the design and implementation of three critical components:

- **The Arithmetic Logic Unit (ALU):** A combinational circuit responsible for performing binary addition and subtraction on data from the Accumulator and B-Register.
- **The Instruction Register (IR):** A temporary storage unit that holds fetched commands from RAM, splitting them into Opcode and Operand for processing.
- **The Controller:** The system's central regulator that decodes the 4-bit opcode from the IR and generates synchronized 12-bit control words to manage the Fetch and Execute cycles.

Together, these components enable the SAP-1 to interpret software commands and convert them into precise electrical signals and arithmetic operations.

Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the computational heart of the SAP-1 (Simple-As-Possible 1) computer. Its primary role is to perform the actual data processing required by the instruction set; specifically, binary addition and subtraction. In this architecture, the ALU is strictly a combinational circuit, meaning it does not store data itself but rather processes the values currently held in the Accumulator (Register A) and the B-Register.



By utilizing 2's complement logic, the ALU can perform subtraction using the same hardware as addition, making it an efficient and elegant solution for an 8-bit breadboard or simulated computer. This module also features bus-interfacing capabilities, ensuring that the result of a calculation is only released to the bus when commanded by the controller. This introduction covers the design and implementation of an 8-bit ALU using standard 74-series TTL logic.

IC Components Used:

To implement the 8-bit Arithmetic Logic Unit (ALU), the following integrated circuits (ICs) and simulation components were utilized within the Proteus environment:

1. **74LS283 (4-Bit Binary Full Adder with Fast Carry):**

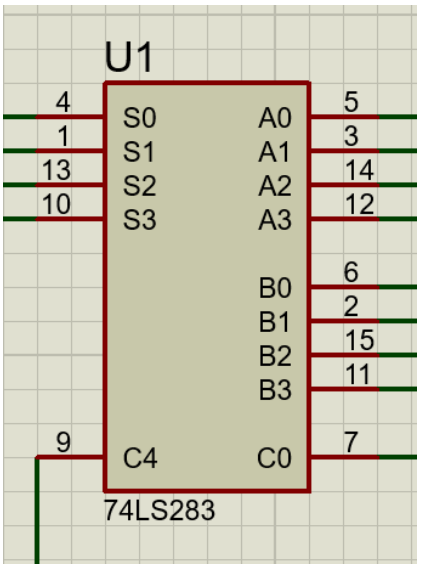


Fig: 74LS283

- **Quantity:** 2
- **Role:** These are the core arithmetic engines. By cascading two 4-bit adders (connecting the Carry-Out of the first to the Carry-In of the second), the ALU achieves full 8-bit addition. It uses internal look-ahead carry logic to minimize propagation delay.

2. **74LS86 (Quad 2-Input Exclusive-OR Gates):**

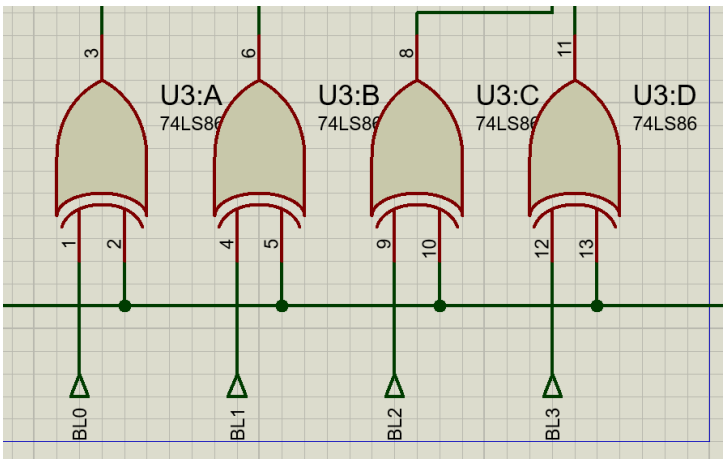


Fig: 74LS86

- **Quantity:** 2
- **Role:** These serve as "Programmable Inverters" for the B-Register bits. When the SU signal is High, the XOR gates flip the bits of B (1's complement). When SU is Low, the bits pass through unchanged.

3. 74LS125 (Quad Bus Buffers with 3-State Outputs):

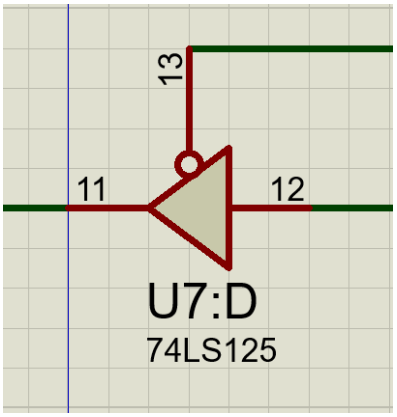


Fig: 74LS125

- **Quantity:** 2
- **Role:** These manage the interface between the ALU and the SAP-1 W-Bus. They prevent "bus contention" by keeping the ALU output in a High-Impedance (Hi-Z) state until the **EO** (Enable Output) signal is triggered.

4. 74LS04 (Hex Inverter):

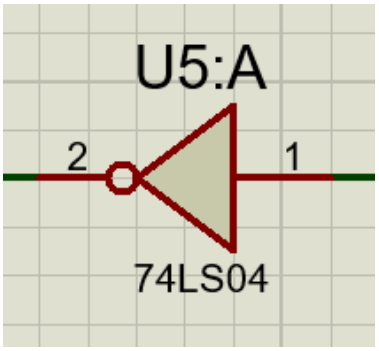


Fig: 74LS04

- **Quantity:** 1 (only 1 gate used)
- **Role:** The 74LS125 buffers utilize an active-low enable. Since the SAP-1 controller typically outputs an active-high EO signal, this NOT gate is used to invert the signal so that a '1' from the controller actually activates the buffers.

5. **Logistates and Logic Probes (Simulation Tools):**

   Role: Used within the Proteus environment to manually toggle the 8-bit inputs for Registers A and B, set control flags (SU/EO), and provide real-time visual verification of the 8-bit result.

Description of Connections

The schematic is divided into a Parent Sheet (Top-level interface) and a Child Sheet (Internal circuitry). The connections are described below:

Parent Sheet (Top-Level Interface): The ALU acts as a processing submodule with defined input and output ports:

- **Inputs (A & B):** The module accepts two 8-bit data words.
  - **AL0–AL7:** Represents the input from the Accumulator (A Register). These are arranged from LSB (AL0) to MSB (AL7).
  - **BL0–BL7:** Represents the input from the B Register. These are similarly arranged from LSB to MSB.
- **Control Signals:**
  - **SU (Subtract):** A generic control line. When set to Logic 0, the ALU performs addition. When set to Logic 1, it performs subtraction.
  - **EO (Enable Output):** Controls the flow of data to the bus. When Logic 1, the result is placed on the bus; when Logic 0, the output is disabled (High Impedance).
- **Outputs (AS0–AS7):** The 8-bit result (Sum or Difference) is output through these ports, which would physically connect to the SAP-1 Bus.

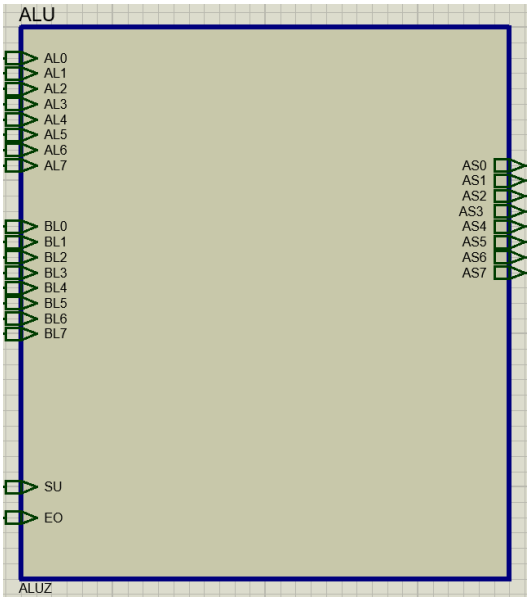


Fig: Parent Sheet Structure of ALU

Child Sheet (Internal Circuitry)

The internal logic is organized into three distinct stages: Input Conditioning, Arithmetic Processing, and Output Buffering.

1. **Conditioning Stage (XOR Array):**
    - The 8 bits of the A-input (AL0–AL7) traverse the circuit directly to the A inputs of the adder ICs.
    - The 8 bits of the B-input (BL0–BL7) are first routed through the 74LS86 XOR gates (U3 and U4). The second input of every XOR gate is tied to the common SU (Subtract) control line. The outputs of these XOR gates connect to the B inputs of the adder ICs.
2. **Arithmetic Stage (Cascaded Adders):**
    - Two 74LS283 ICs (U1 and U2) are used. U1 handles the lower nibble (bits 0-3), and U2 handles the upper nibble (bits 4-7).
    - **Carry Propagation:** The Carry Out (C4) pin of the lower nibble adder (U1) is connected directly to the Carry In (C0) pin of the upper nibble adder (U2), allowing the carry bit to ripple through to the higher bits.
    - **SU Connection:** The SU control line is connected to the Carry In (C0) pin of the LSB adder (U1). This is crucial for 2's complement subtraction (providing the necessary "+1").
3. **Output Stage (Tristate Buffers):**
    - The sum outputs (S0–S3) from both adders are routed to the inputs of the 74LS125 buffers (U7 and U8).
    - The EO control signal passes through a 74LS04 NOT gate (U5:A). The output of this inverter drives the active-low Enable pins (~E) of all eight buffers. This ensures that when the external EO is High, the buffer enable pins are pulled Low, activating the output.

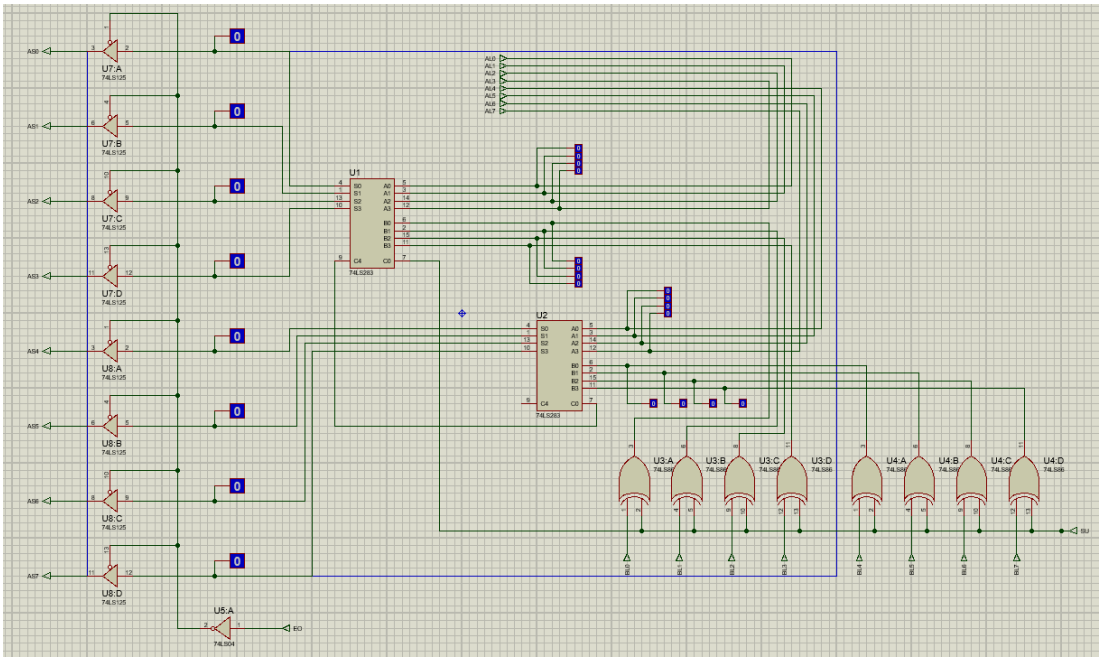


Fig: Child sheet of ALU

Functions

The designed ALU module performs three primary functions within the SAP-1 architecture:

1. **Binary Addition:** Adds the 8-bit value of the Accumulator to the 8-bit value of the B-Register (A + B).
2. **Binary Subtraction:** Subtracts the B-Register value from the Accumulator value using 2's complement logic (A - B).
3. **Bus Isolation:** Utilizes tristate buffers to disconnect the ALU from the computer's common bus when the ALU is not the active source of data, preventing bus contention.

Working Principle

The operation of the ALU relies on the status of the SU (Subtract) and EO (Enable Output) control lines.

Addition Mode (SU = 0)

When the SU line is Low (0):

- The XOR gates receiving the B-register bits act as non-inverting buffers (since B XOR 0 = B). The B data reaches the adders unchanged.
- The Carry In (C_0) of the first adder (U1) is 0.
- The 74LS283 chips perform standard binary addition: Result = A + B + 0.

Subtraction Mode (SU = 1)

When the SU line is High (1), the circuit utilizes 2's complement arithmetic to perform subtraction (A - B is processed as A + (-B)).

- **1's Complement:** The XOR gates act as inverters (since B XOR 1 = ~B). The B inputs at the adders are now the inverted version of the original data.
- **2's Complement:** The SU line is also connected to the Carry In (C_0) of the first adder. This adds a '1' to the total sum.
- **The Calculation:** The adder calculates A + ~B + 1. In binary arithmetic, ~B + 1 is the 2's complement (negative representation) of B. Therefore, the result is effectively A - B.

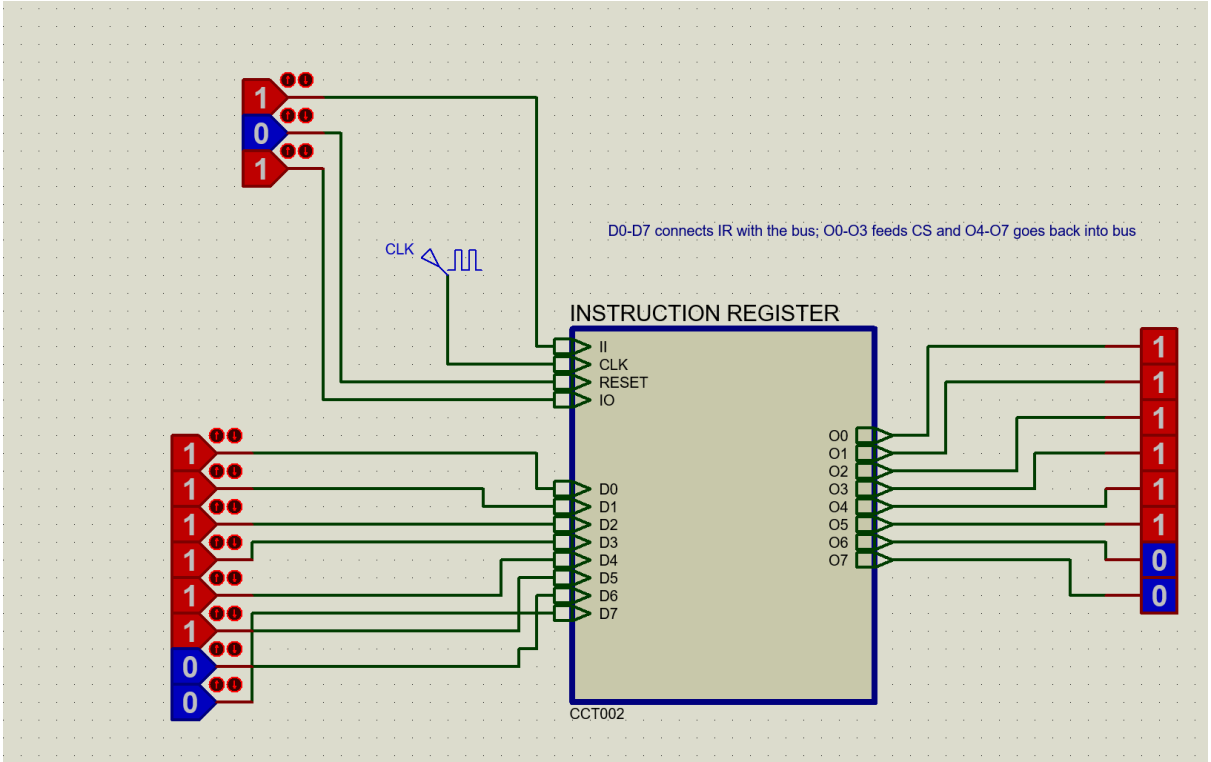
Output Control (EO)

Regardless of the arithmetic operation, the data must be gated to the bus:

- If **EO = 0**: The inverter outputs a High signal to the 74LS125 enable pins. The buffers enter a High-Impedance state, effectively disconnecting the ALU from the bus.
- If **EO = 1**: The inverter outputs a Low signal to the 74LS125 enable pins. The buffers become active, and the result of the addition/subtraction is placed onto the output pins (`AS0`–`AS7`).

Instruction Register:

In SAP-1 architecture the ‘Instruction Register’ works as a temporary storage which is being performed and acts as a linkage between RAM and CU.

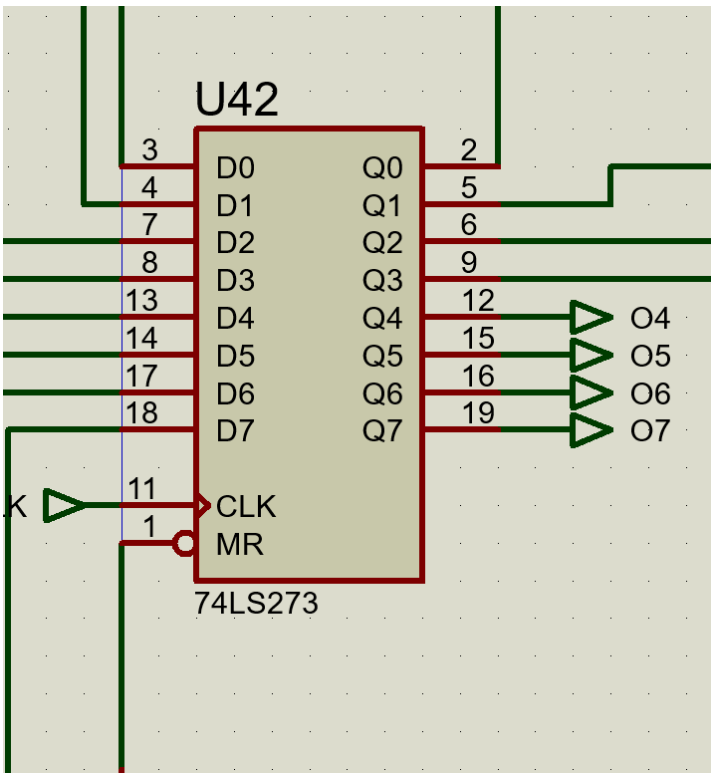


The primary role of the Instruction Register is to receive an instruction from the RAM during the Fetch Cycle and hold it while the computer understands and performs this instruction.

Unlike other registers that hold data to be used for arithmetic e.g Accumulator and addresses e.g. the MAR, the IR holds a command that instructs it what to do next.

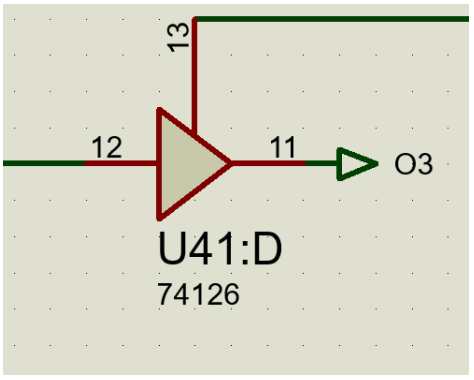
IC Components used in Instruction Register:

74LS273: Octal D-flipflop IC (1)



1. 74LS273 acts as the storage unit for the Instruction Register.
2. This IC is positive edge triggered.
3. Pins D0-D7 are connected to the bus.
4. Master reset is Active low and is used to reset the Instruction Register to default state.
5. Output pin Q0-Q3 are connected to the controller through the sub-circuit and Q4-Q7 are connected back to the bus.

74126: Tristate buffer (12)

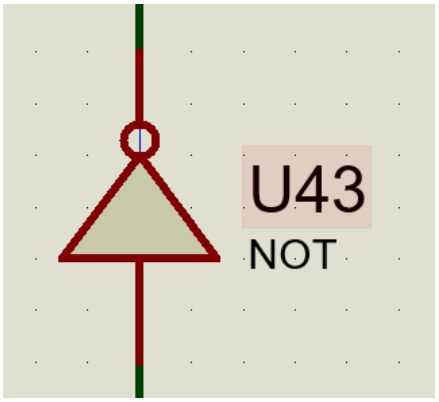


1. 12 Tristate buffers are used to control whether the IR will accept the inputs from the bus and output will be delivered to the bus and controller.

2. 8 (D0-D7) tristate buffers on the input pins control if IR will accept the input from the bus.

3. 4 (Q0-Q3) tristate buffer on the upper four output nibble controls whether the output will be delivered to the Controller.

NOT Gate IC: Not gate simulation primitive IC



A single not gate IC was used to invert the input to Master Reset as the MR was active low.

Logic Connections and Control Lines:

1. **Clock Line:** Connected to activate the IC in the positive edge of the clock signal.
2. **Load Line:** Used for the connection to bus
3. **Reset:** Used to reset the IR.
4. **Instruction In:** Used to control data input into the IR.
5. **Instruction Out:** Used to control data output from the IR to the controller.

Connection Description:

Parent Sheet:

Input Pins:

D0-D7: 8 inputs from the bus is introduced to the IR through input pins D0-D7.

CLK: The clock signal generated from the Clock Pulse generator connects to the octal D-flipflop IC.
II: This connection provides the user with the ability to freely control if data enters the IR.

IO: Similar to II, IO allows the user the restrict if data will be sent to the controller.

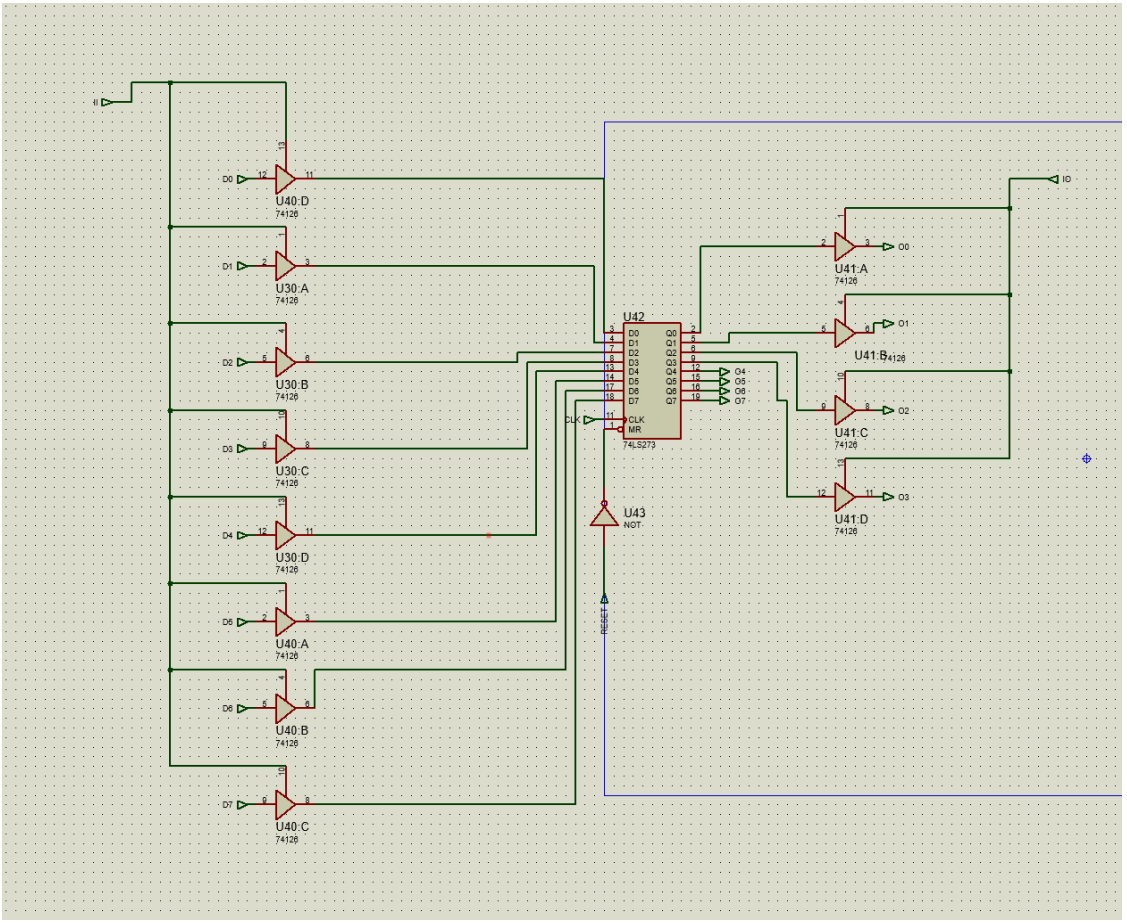
Output Pins:

O0-O3: These four connections are used to send data to the controller.

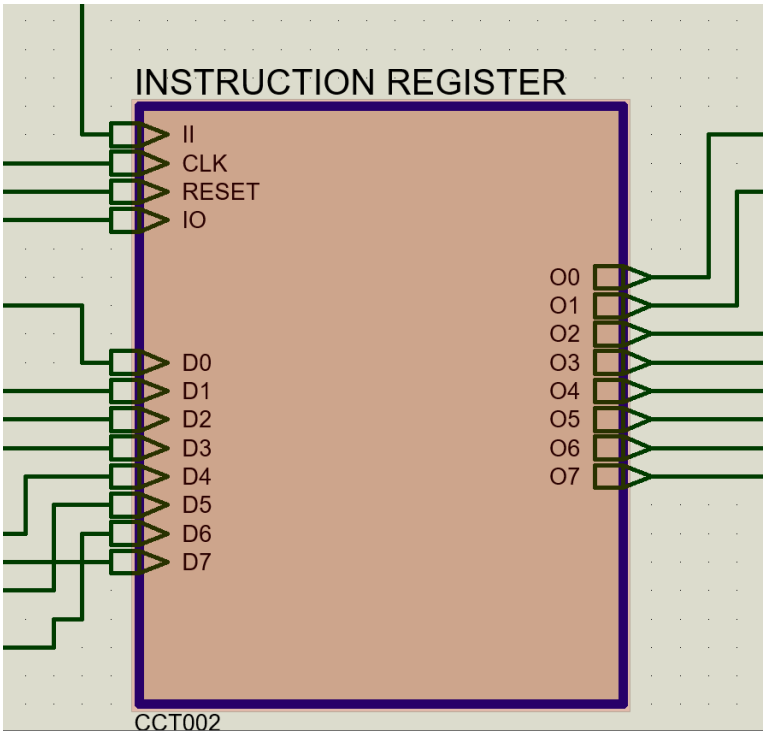
O4-O7: At last, these four connections are used to send data back to the bus.

Connections inside Child Sheet:

1. D0-D7 of the parent sheet is connected to D0-D7 of 74LS273 IC via eight tristate buffers and the controlling inputs of the buffers are shorted with II.
2. Q0-Q3 are connected to O0-O3 again via four tristate buffers and the controlling inputs of the buffers are shorted with IO.
3. Q4-Q7 are normally connected to O4-O7.
4. Clock input from the parent sheet was connected to the clock input of 74ls273 IC.
5. The input of reset from parent sheet was inverted as the MR of 74ls273 was active low.



Child Sheet of Instruction Register



Parent Sheet of Instruction Register

Functions:

The Instruction Register (IR) is where the command processing occurs in SAP-1. During the fetch portion of the cycle, the instruction is brought in from the RAM to the bus. The IR generates its load line to capture and store the 8 bit word in its instruction register. The IR is responsible for holding the instruction stable while the computer processes it so that the instruction does not disappear from the bus during processing.

The IR separates the 8 bit command instruction into two 4 bit commands for completely different purposes. The high 4 bits (O0 - O4) are called the Op Code (Operation Code) and they are connected to the Controller and will never return to the Main Data Bus. The Controller will read the Op Code and use this information to decide what to do next (add, subtract, load, etc.) and then generate control signals needed to complete the selected task.

The low 4 bits (4-7) are called the Operand and they most generally hold a Memory Address. The Operand works with the Main System Bus while the Op Code does not.

Working Principle:

One of the important functions of the Instruction Register (IR) in SAP-1 is to act as a temporary storage and a means of allocation centre for the instructions during the clock cycle. In this context, an Instruction Register works by capturing an 8-bit instruction from the system bus and dividing that single instruction into two functional 4-bit nibbles (the Operand and Opcode). By splitting the instruction into its two functional parts, the Control Unit can determine precisely what function needs to be executed, and the remaining portions of the machine can locate the necessary data to accomplish that execution.

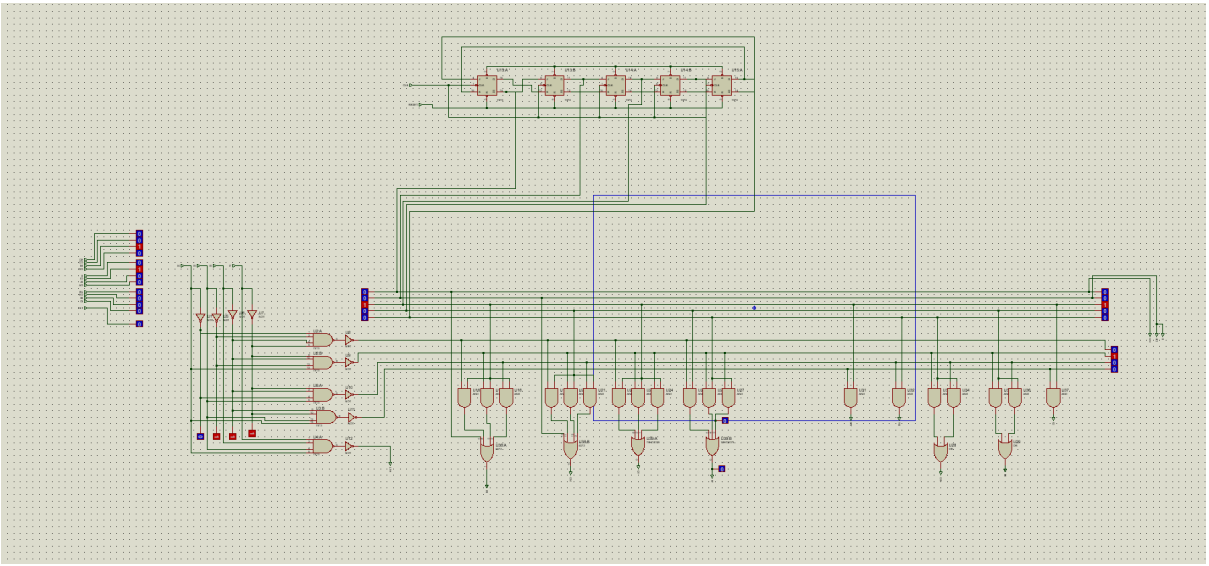
The IR begins its primary function during the third state of the fetch cycle, also known as State T3. At this point in time, the Program Counter has identified the memory address to be used, and the corresponding 8-bit instruction has been placed on the bus by the RAM. The Controller generates a low LI (Load Instruction) signal that indicates to the IR to latch the currently available 8-bit

word from the bus. Once the signal has gone back to high, the instruction is locked into the IR, remaining stable and available for use even though the data on the bus continues to change.

Once the 8-bit instruction has been stored in the IR, the internal circuitry of the IR treats the 8-bit instruction as two distinct parts when decoding and executing.

Controller:

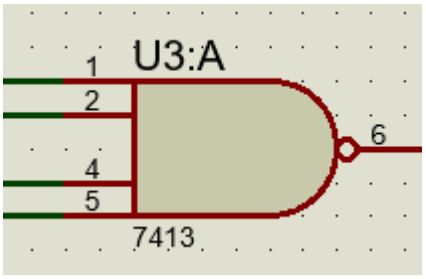
The Controller is the central processing unit of the SAP-1 architecture as it controls all the data transfer flows and all instruction processing. The system works by decoding the 4-bit opcode obtained from Instruction Register, which is synchronized with a ring counter measuring T-state time intervals from T1 to T5.



The controller generates 12 distinct Control signals in its clock pulse operation by processing multiple inputs to generate output signals that will activate other system components using their "Enable" and "Load" activation functions. Data transfer onto the bus and from the bus happens at precisely the right time, which allows the computer to go through its Fetch cycle and Execution cycle, depending on the instruction.

IC Components used for Controller:

7413: Quad Input Nand Gate (5)



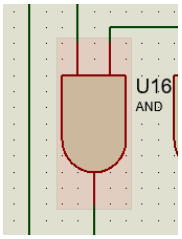
Usage: Five of such quad input and gates were used to generate the four distinct operations (Load, Add, Subtract, Halt, Output), using the four input signals.

Not Gate: Simulation Primitive Not Gate (9)



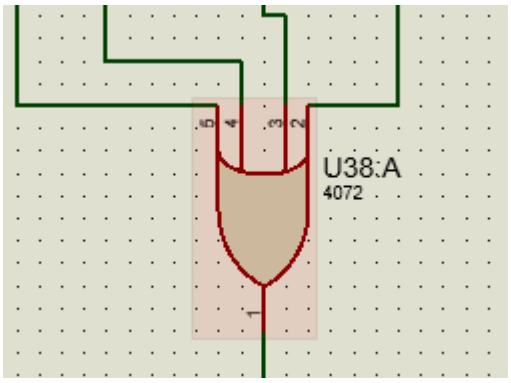
Usage: Four of such gates were used to generate the complement of the input signals (I4-I7) and rest five were used to invert the outputs of the 7413 nand gate.

And Gate: Simulation Primitive Dual Input And Gate (19)



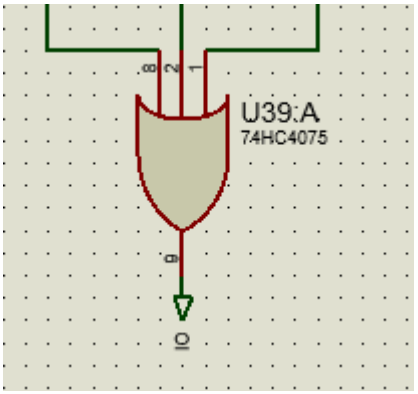
Usage: Nineteen of such dual input and gates were used to generate the nine distinct control signals. The inputs either came from the ring counter or the instruction channel.

4072: Quad Input Or Gate (2)



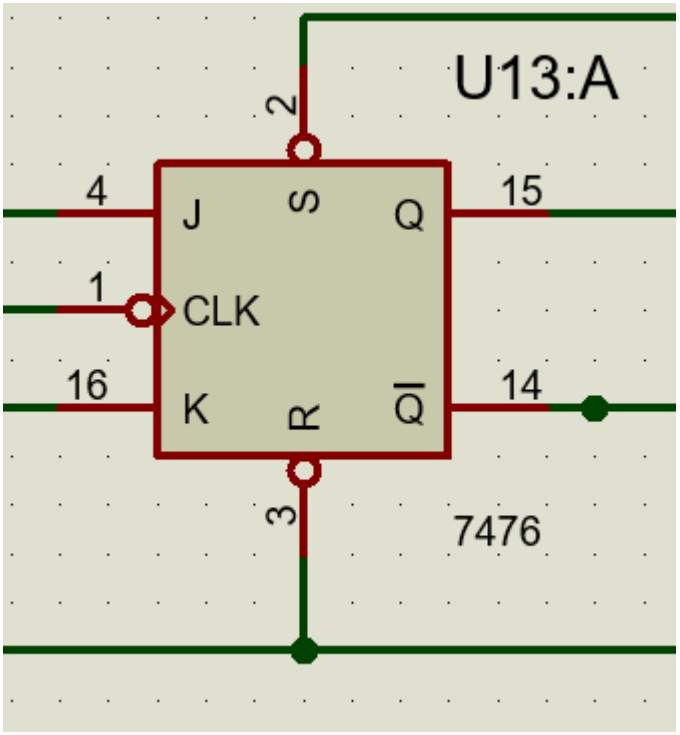
Usage: Two such gates were used to combine the two sets of four outputs generated by the and gates.

74HC4075: Tri-input Or Gate (2)



Usage: Two tri-input or gates were used to combine the two sets of three outputs generated by the and gates.

7476: Dual JK Flipflop with Set and Reset (5)



Usage: Five Dual JK flipflops were so connected to obtain a ring counter that was used as trigger states. In each trigger state certains control signals were enabled using and gates.

Description of Connection:

Parent Sheet:

Input Pins:

I4-I6: Four inputs from the Instruction register are fed through these four pins. These four pins represent the upper four nibble of the instruction.

CLK: The clock signal is supplied through this pin.

Reset: This pin controls the value current displayed by the ring counter.

Output Pins:

CE: This pin controls the clock enable pin.

CO: The output enable pin of ‘Program Counter’ is controlled by this pin.

MI: This pin controls whether the address will be accepted by the ‘MAR’.

RO: Controls whether the output of ‘RAM’ will be delivered to the bus.

II: Controls if the instruction will be written on the ‘Instruction Register’.

IO: Controls if the lower four nibbles of instruction will be sent to the bus.

AI: Controls if the then values on the bus will be saved into Accumulator.

AO: Controls if the then values of the Accumulator will be sent to the bus.

SU: Used to select between Subtraction (High) and Addition (Low).

EO: Used to enable if the output of ALU will be sent to the bus.

BI: Controls if the then values of the bus will be saved into B Register.
OI: Used to control whether the then values of the bus will be stored in the Output Register.

HLT: Used to halt the process of the computer by stopping the main clock.
Child Sheet:

I4-I7: The upper four nibble of the instruction from the Instruction Register is connected through I4-I7.

CLK: Clock pins of the five JK Flipflop are shorted with the clock input.

Reset: The reset pin of the five JK Flipflop is shorted with the reset input.

Connections of The Ring Counter:

1. The Q input of the first is fed to the K of the second and Q’ to J.
2. For the next three JK flipflops the Qs were connected to the Js and the Ks were connected to the Q’s.
3. Lastly for the last JK flipflop the Q input of the last is fed to the K of the first and Q’ to J.

Connection of the Output Signals:

CE: Output is taken from T1.

CO: Active in T-1 cycle.

MI: Active in T-1 cycle and during addition, subtraction and loading.

RO: Active in Trigger-2 cycle and during addition, subtraction and loading.

II: Active in T-1 cycle.

IO: Active during addition, subtraction and loading.

AI: Active during addition, subtraction and loading.

AO: Active during T-3 cycle alongside output command.

SU: Active during T-5 cycle with the subtract signal.

EO: Active during T-5 cycle with the subtract and add signal.

BI: Active in T-4 cycle with the subtract and add signal.
OI: Active in T-4 cycle.

HLT: Four upper nibbles (I4-I7) anded together.

Instruction Name	Codename	Opcode
Load	LDA	0000
Add	ADD	0001
Substact	SUB	0010
Output	OUT	0011
Halt	HLT	1111

Function:

The function of the controller is to serve as the automated regulator of the computer's architecture, decoding program instructions and controlling the flow of data between components.

It functions by analyzing the 4-bit opcode received from the Instruction Register and synchronizing it with a ring counter's timing signals to generate a 12-bit control word for every clock cycle. By doing so, the controller ensures that the Fetch cycle and the Execute cycle happen in the correct sequence, effectively translating abstract software commands into physical electrical signals that drive the hardware.

Working Principle:

The controller in the SAP-1 architecture acts as the central nervous system, controlling precise timing and sequence of data movement by generating control signals based on the current instruction and a timing counter.

It operates by taking the 4-bit opcode from the Instruction Register and combining it with timing signals (T1 through T65) generated by a ring counter, which divides the machine cycle into six distinct states. During the first three states (T1–T2), the controller outputs a fixed sequence of signals to perform the "Fetch" cycle, moving the instruction from memory to the Instruction Register. During the execution states (T3–T5), the controller's logic circuit decodes the specific opcode and activates the unique combination of control lines needed for the smooth execution of that specific command.

Discussion

In Phase C of our project, we focused on designing and implementing the core processing and control units of the 8-bit SAP-1 computer: the Arithmetic Logic Unit (ALU), the Instruction Register (IR), and the Controller. This phase was pivotal as it transitioned our work from building static storage and timing modules to constructing the active computational and decision-making brain of the system.

ALU Design and Implementation: We designed the ALU as a strictly combinational circuit capable of performing binary addition and subtraction. By utilizing 2's complement logic, we were able to perform subtraction using the same hardware as addition, which proved to be an efficient design choice for our simulation. We used two 74LS283 4-bit full adders cascaded together to handle 8-bit operations , and a bank of 74LS86 XOR gates to act as programmable inverters for the B-register input, enabling the subtraction capability when the SU signal is high. A key learning point here was managing bus contention; we integrated 74LS125 tristate buffers to ensure the ALU only places data on the bus when the "Enable Output" (EO) signal is active.

Instruction Register and Decoding: The Instruction Register (IR) was implemented using a 74LS273 octal D-flip-flop to temporarily store instructions fetched from RAM. We learned to split the 8-bit instruction into two distinct 4-bit parts: the upper nibble (Opcode) which is sent to the Controller, and the lower nibble (Operand) which typically holds an address. This separation is crucial for the Fetch and Execute cycles. We also incorporated tristate buffers to control the flow of data into and out of the IR, allowing us to isolate the instruction from the bus once it is fetched.

Controller and Timing: The Controller serves as the "central nervous system" of our architecture. We designed it to decode the 4-bit opcode from the IR and synchronize it with timing signals (T1-T6) from a ring counter. By using a combination of AND, OR, and NAND gates, we generated the precise 12-bit control words needed to orchestrate the movement of data between registers during each clock cycle. This part of the phase deepened our understanding of how abstract software commands (like LDA, ADD, SUB) are translated into physical electrical signals that drive hardware components.

Overall Insights: Through this phase, we gained significant practical insight into the complexity of coordinating parallel hardware events. We successfully integrated the ALU's arithmetic capabilities with the Controller's command sequencing, ensuring that operations like "Add" or "Subtract" occur only at the

correct timing step (e.g., T5). This phase highlighted the importance of precise timing and signal buffering in digital system design, setting a strong stage for the final integration of the complete computer.

Credits:

Work Contribution	Contributors
Arithmetic Logic Unit (ALU)	Rafin
Instruction Register (IR), Controller Sequencer	Raad
Report writing	Rafin and Raad